

Task 4b - Code review

Review of how my code met the system requirements.

Overall I believe that I have produced a solution that meets all the system requirements to a good standard.

The first thing I did was to add an initial menu that provides the user with three options, one for each of the main system requirements given in the task.

In this section I ensured the code was consistent and robust by refactoring some the code provided in the original code file.

```
def main_menu():
    flag = True

    while flag:
        print("#####")
        print("Welcome to the RBSX currency analysis tool")
        print("Please select an option")
        print("1. Calculate currency conversion")
        print("2. Compare GBP to other currencies")
        print("3. Currency performance analysis")
        print("")
        print("#####")

        menu_choice = input("Please enter the number of your choice (1-3): ")

        try:
            int(menu_choice)
        except:
            print("Sorry, you did not enter a valid choice")
            flag = True
        else:
            if int(menu_choice) < 1 or int(menu_choice) > 3:
                print("Sorry, you did not neter a valid choice")
                flag = True
            else:
                return menu_choice
```

I adapted the range of the input check from the original code so that it provided appropriate validation for the user input.

Next I restructured some of the code to fit with the approach I took with the initial menu.

The lines of code that I have grouped under main_choice 1 were originally spread out throughout the code base and were manipulated in the global space. I chose instead to keep them contained within the main() function and pass variables between them, as such my code follows secure coding practices helping me meet the requirement for “must be secure”

```
def main():
    main_choice = main_menu()

    if main_choice == "1":
        menu_choice = menu1()
        currency = get_currency(menu_choice)
        conversion_rate = get_conversion_rate(currency)
        conversion_amount = float(get_amount_to_convert(currency))
        perform_conversion(conversion_amount, conversion_rate, currency)
```

I then addressed the first additional requirement I addressed was showing trends and patterns the value of the pound (GBP) compared to other currencies.

To meet this requirement, I organised my code into three main processes which are contained within functions.

Firstly, the code gets the user's choice of currency that they wish to view and returns a string that can be used to identify the hearer in the dataframe.

```

def currency_performance():
    flag = True

    while flag:
        print("Please select the currency you wish to see the performance of:")
        print("1. Euros (EUR)")
        print("2. Australian Dollars (AUD)")
        print("3. Japanese Yen (JPY)")
        print("#####")

        compare_choice = input("Please enter the number of your choice (1-3): ")

        try:
            int(compare_choice)
        except:
            print("Sorry, you did not enter a valid choice")
            flag = True
        else:
            if int(compare_choice) < 1 or int(compare_choice) > 3:
                print("Sorry, you did not enter a valid choice")
                flag = True
            else:
                flag = False

        if compare_choice == "1":
            currency_header = "EUR - GBP"
        elif compare_choice == "2":
            currency_header = "AUD - GBP"
        else:
            currency_header = "JPY - GBP"

    return currency_header

```

I think that this code is a little inefficient, as the headings are hard coded. I did start to develop a function that extracted the header row from the CSV, which would make the code more scalable, however I was not sure I could get it working fully in the time I had, so I chose instead to prioritise functional over efficient code.

This is the start of the alternative code I was going to use:

```

#extracts the headers from the csv file
def get_currency_headers():
    data = pd.read_csv("Task4a_RBSX_data.csv")
    currencies = list(data.columns.values)
    currencies.pop(0)
    return currencies

```

The second part of the code, for this gets the user to choose the time frame that they wish to see the comparison for.

I made the code easy to use by providing three options, for which the user only has to enter the relevant number, this reduces the chance of user error, and the simplicity of this ensures a good user experience.

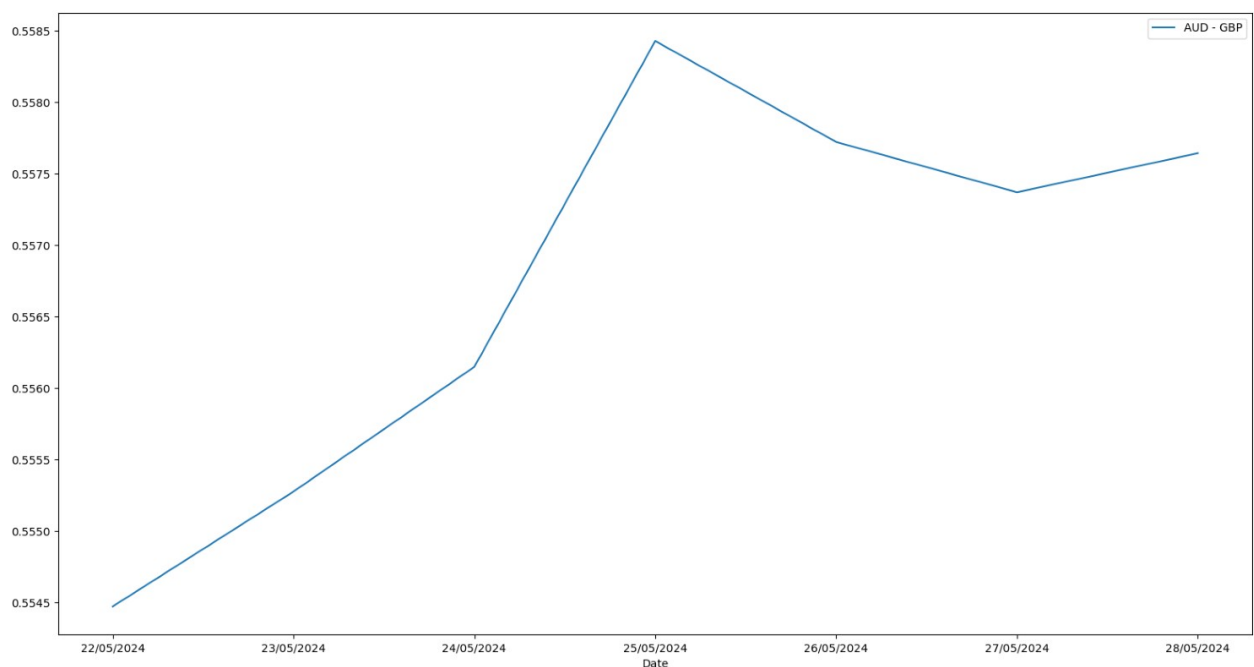
```
def get_compare_range():  
    flag = True  
  
    while flag:  
        print("#####")  
        print("##### Historical data #####")  
        print("#####")  
        print("Please choose a time fram for your comparison")  
        print("1. Last 7 days")  
        print("2. Last 14 days")  
        print("3. Last 30 days")  
        print("#####")  
  
        range_choice = input("Please enter the number of your choice (1-3): ")  
  
        try:  
            int(range_choice)  
        except:  
            print("Sorry, you did not enter a valid choice")  
            flag = True  
        else:  
            if int(range_choice) < 1 or int(range_choice) > 3:  
                print("Sorry, you did not neter a valid choice")  
                flag = True  
            else:  
                return range_choice
```

I chose to provide three predefined time options, 7 days, 14 days and 30 days which provides the last 7,14 and 30 entries in the data file. I chose to do this because the first part of the requirements was to use “most up to date figures”. Although this was only stated in the first requirement and related to the code that was provided, I thought using more recent data is more relevant to a user’s needs.

The final part of the is functionality, uses the iloc method from pandas to extract the required data from the initial data frame. I chose to output this data in both graphical and tabulated form to meet a range of user needs.

I also provided an average value of the conversion rate over the given time period.

```
def process_value_data(range,currency_header):  
  
    main_df = pd.read_csv("Task4a_RBSX_data.csv")  
  
    if range == "1":  
        gbp_df = main_df[["Date",currency_header]]  
        seven_day_gbp = gbp_df.iloc[-7:]  
        seven_day_avg = round(seven_day_gbp[currency_header].mean(),3)  
        seven_day_gbp.plot(x="Date", y = currency_header)  
        plt.show()  
        print("GBP to {} for the last 7 days".format(currency_header))  
        print(seven_day_gbp)  
        print("")  
        print("The aveage convesion rate for the last 7 days is: ", seven_day_avg )
```



```
##### Historical data #####  
#####
```

```
Please choose a time fram for your comparison
```

1. Last 7 days
2. Last 14 days
3. Last 30 days

```
#####
```

```
Please enter the number of your choice (1-3): 1
```

```
GBP to AUD - GBP for the last 7 days
```

	Date	AUD - GBP
79	22/05/2024	0.554472
80	23/05/2024	0.555276
81	24/05/2024	0.556149
82	25/05/2024	0.558428
83	26/05/2024	0.557722
84	27/05/2024	0.557370
85	28/05/2024	0.557643

```
The aveage convesion rate for the last 7 days is: 0.557
```

The second first additional requirement I addressed was showing trends and patterns for the performance (e.g. increase/decrease in value) of a selected currency.

As with the first additional requirement, I broke the code into three main processes. The first two were very similar in that they were focused on getting the user's choices.

The final part then extracts the data for the chosen currency over the chosen time frame.

In order to show how the currency performed, I extracted the first and last value from the subset of data, and then subtracted the initial value from the final value to see if there had been an increase or not.


```

def process_performace_data(range,chosen_currency):

    main_df = pd.read_csv("Task4a_RBSX_data.csv")

    if range == "1":
        currency_df = main_df[["Date",chosen_currency]]
        seven_day_perf = currency_df.iloc[-7:]
        start = seven_day_perf[chosen_currency].iloc[1]
        end = seven_day_perf[chosen_currency].iloc[-1]
        variation = end - start
        print("The starting value of this currency compared to GBP is:",start)
        print("The final value of this currency compared to GBP is:",end)
        if variation > 0:
            print("This currency has increased in value over the last 7 days")
        elif variation < 0:
            print("This currency has decreased in value over the last 7 days")
        else:
            print("This currency has not changed in value over the last 7 days")

```

How I addressed the user requirements

As well as meeting the system requirements, I believe that I have met the user requirements for this program.

Firstly, I made it easy to use by continuing the existing menu system. The menus use coding so that the user only has to enter a single number to choose the option that they wish to see. The menus also provide helpful messages if the user enters an incorrect option.

Secondly, the code displays information in a meaningful way, I have used a mix of terminal output and the plotting functionality in matplotlib to provide graphs of the data. Providing data in both formats, allows users to explore the data in a format that best suits their preferred style.

How the solution could be further developed

As I mentioned earlier in my review, there are some parts of the code that are a little inefficient. For example, the `currency_performance()` function has hard coded options for the three different currencies. If I was to develop this, I would add functionality that extracts the header rows from the csv file and uses this to both provide a list of options and retrieve the strings for passing to other functions. By doing this it would allow the code to be more scalable as it would be able to use a CSV file that contains a range of different currencies, providing a more useful tool for the company.

The second improvement I would make is to provide a greater flexibility for the user when selecting the time frame they wish to analyse. I would do this by allowing the user to select the specific number of days that they wish to see. I think this would also allow me reduce the amount of code in each of the main data processing functions as I could use a variable to store the number of days provided by the user and pass it to a single piece of code, rather than having 3 blocks of code that contain a lot of repetition.

Another option could be to allow the user to select a start and end date for the analysis, rather than by a number of days.